PT

Incremental Python Test Engine

Implementation specification, work packages,
and a benchmark protocol built to resist self-deception.

Fresh execution. Persistent preparation.
Real pytest as the initial semantic backend.
Explicit contracts before aggressive optimization.
Evidence before speed claims.

20	12	3
WORK PACKAGES	CORRECTNESS INVARIANTS	BENCHMARK CLAIM FAMILIES

7 November 2026
Ready for engineering review · Proposed architecture

No implementation, compatibility pass rate, or performance result is claimed. Numerical targets are proposed engineering gates. `pt` is a working name; availability has not been checked.

Contents

The canonical editable source is the accompanying Markdown specification. All proposed command interfaces and acceptance thresholds require implementation and measurement.

1. Product thesis and scope 3
2. Correctness contracts 4
3. Architecture and lifecycle 5
4. Identity, plans, and persistent storage 6
5. Source indexing and semantic resolution 8
6. Capabilities and legal transformations 8
7. Process supervision and warm preparation 9
8. Scheduling, fixtures, and managed resources 10
9. Incremental selection and source-generation integrity 10
10. Event protocol and reporting 11
11. User interface, packaging, and failure policy 12
12. Benchmark charter: what a claim is allowed to mean 13
13. Baselines and fair configuration 13
14. Benchmark corpus 15
15. Cache and lifecycle state matrix 16
16. Harness design and experimental procedure 17
17. Metrics and analysis 19
18. Coverage and diagnostic equivalence 21
19. Incremental-selection evaluation 21
20. Publication contract and benchmark self-tests 23
21. Work packages 24
22. Dependency roadmap and release gates 31
23. Risk register and kill criteria 32
24. Decisions still requiring implementation evidence 33
Appendix A. Repository layout and core records 34
Appendix B. Benchmark scenario template 35
Appendix C. Sample and result-card layouts 36
Appendix D. Primary references and source boundary 37

Reader's guide

Sections 1–11 define the product and architecture. Sections 12–20 define the benchmark contract, experimental protocol, and release evidence. Section 21 contains twenty dependency-ordered work packages. Sections 22–24 define delivery gates, risks, and open decisions. Appendices provide record layouts, a scenario template, and primary references.

The words **MUST**, **MUST NOT**, **SHOULD**, and **MAY** express requirements within this proposal. Numeric targets are proposed engineering gates, not measured achievements or universal statistical guarantees. All example `pt` and `pt-bench` commands describe interfaces to implement, not tools supplied with this document.

1. Product thesis and scope

1.1 The product

PT is a local-first engine for running existing pytest suites with less repeated preparation, better permitted scheduling, and faster edit–test feedback. It separates three objects:

- **Execution plan:** a versioned description of requested work and its provenance.
- **Execution contract:** the transformations permitted for that work.
- **Execution session:** the process-owned mutable Python state in which work actually runs.

Persist descriptions and observations. Discard executed application state at the end of its session. A fresh session does **not** mean a fresh process for every test: tests within a normal pytest session retain normal fixture and module lifetimes.

The initial product is not a native reimplementation of pytest. Rust owns orchestration and persistent metadata; pytest remains responsible for Python test semantics. This deliberately allows an early version to be approximately as fast as, or slightly slower than, direct pytest. The first milestone proves a trustworthy boundary, not a marketing speedup.

1.2 Goals and non-goals

Priority	Goal	Evidence required
P0	Preserve the selected backend's supported semantics	Differential oracle, adverse cases, explicit unsupported cases
P0	Never report stale or incomplete execution as full success	Generation checks, phase accounting, crash tests
P0	Measure end-to-end cost honestly	Independent harness, lifecycle-inclusive timing, raw evidence
P1	Reduce repeated preparation where safe	Controlled cache/preload ablations and equal-work comparisons
P1	Improve local feedback with affected-test selection	Mutation/replay audits, separately labelled workflow results
P1	Explain every optimization and fallback	Machine-readable decisions and `pt explain`
P2	Improve parallel scheduling and resource utilization	Equal-resource comparison with well-configured xdist
Research	Replace measured Python orchestration bottlenecks	Native-backend conformance plus a separate go/no-go decision

V1 explicitly excludes universal plugin equivalence, arbitrary Python heap serialization, transparent reuse of executed interpreters, distributed remote execution, remote result caching, a replacement coverage engine, and sandboxing hostile tests. Result reuse is not test execution and is out of scope for the initial release.

1.3 Position relative to existing projects

The reviewed projects motivate four distinct approaches: pytest-rs advertises a Rust runner, rpytest emphasizes warm runtime/startup behavior, rupt uses a Rust-oriented runner and discovery approach, and xdist distributes pytest sessions. These descriptions establish the design space, not an independently verified ranking. Their published speedups MUST NOT become PT's baseline evidence. [S01–S04]

Existing xdist scheduling modes already include scope/group-aware distribution and work stealing. PT MUST benchmark against the relevant modes instead of claiming those capabilities as new. Dependency-informed selection also already exists in testmon and belongs in the workflow comparison set. [S05, S12]

1.4 Initial support proposal

The first conformance target is standard, GIL-enabled CPython on Linux x86-64. The proposed expansion matrix includes CPython 3.12–3.14, macOS arm64, and Windows x64, subject to compatible pinned pytest/plugin versions. This is a proposed bounded support window, not a statement about the latest Python release. Free-threaded builds, alternative interpreters, and project snapshots remain separate experimental profiles until certified.

WP00 freezes the exact matrix. Unsupported versions MUST produce a clear unsupported-profile result or an explicitly requested passthrough execution; they MUST NOT inherit a conformance claim from a nearby version.

2. Correctness contracts

2.1 Three different compatibility claims

Serial compatibility compares PT's pytest backend against a fresh invocation of the same pinned pytest entrypoint, interpreter, arguments, plugin configuration, source, and relevant environment. It preserves collection multiplicity/order, selection, fixture lifecycle, test outcomes, and exit behavior within the supported profile.

Parallel compatibility compares against an explicitly selected parallel contract, normally a pinned xdist configuration. Parallel execution is not advertised as equivalent to serial execution for arbitrary order-dependent tests. Reordering, worker identities, fixture multiplicity, and fail-fast behavior can differ across parallel policies; those differences need explicit permission.

Affected-run behavior executes a subset and reports its scope. It does not establish that the full suite passes. Historical observations and static imports alone do not establish complete dependency knowledge for arbitrary Python programs.

A comparison record MUST name its contract. “Drop-in compatible” without a versioned scope is not a release criterion.

2.2 Non-negotiable invariants

ID	Invariant
C01	A source index is never treated as authoritative pytest collection.
C02	The default backend collects and executes within the same pytest session; no hidden preflight collection run.
C03	A process that has executed project tests is never reused for a later run.

ID	Invariant
C04	Unknown capabilities disable the relevant optimization, not the plugin.
C05	Optimization fallback never automatically replays already-started work.
C06	Setup, call, teardown, collection, and infrastructure failures remain distinguishable.
C07	A completed older generation never marks the current generation green.
C08	Hard timeout means terminating the owned execution boundary, not merely abandoning a wait.
C09	A full-run success requires complete requested-work accounting and successful required finalization.
C10	Test selection, reordering, fixture replication, and resource reuse each require their own permission.
C11	Cache read/validation, reporting, required coverage merge, and required shutdown remain inside the applicable latency boundary.
C12	A benchmark mismatch is a correctness failure, not a faster sample.

Pytest's fixture lifetimes, collection hooks, and plugin loading behavior are the reason to make these boundaries explicit rather than infer safety from a test function's signature. [S06–S08]

2.3 What equivalence does not promise

PT does not promise identical PIDs, exact timestamps, terminal byte ordering under concurrent output, or identical resource usage. It does not guarantee equivalence for plugins that deliberately inspect the presence of PT's bridge or rely on unspecified internal objects. Such cases require an adapter, a documented passthrough profile, or an unsupported classification.

The conformance oracle compares a defined observable contract. Passing a finite corpus is evidence for that contract, not a proof that every possible plugin behaves identically.

3. Architecture and lifecycle

3.1 Component boundary

```
CLI / editor client
        |
        v
Rust coordinator ---------------- persistent metadata store
  | source index / identity       plans, durations, dependency observations
  | capability policy
  | process supervisor
  | event aggregation
  |
  +--> fresh Python session: real pytest + explicitly loaded bridge
  |       collection -> resolution -> execution -> finalization -> exit
  |
  +--> optional pinned xdist backend
  |
  +--> optional audited seed process -> disposable execution children
```

The coordinator MUST NOT import user Python code. The initial implementation uses a subprocess boundary, not embedded CPython inside the Rust daemon. This avoids tying interpreter crashes and mutable application state to the long-lived control plane.

An optional daemon persists coordination and metadata. A no-daemon path MUST remain available for debugging, CI, and recovery.

3.2 Default serial execution sequence

1. Resolve the user-selected interpreter and invocation profile without importing the project.
2. Capture request arguments, relevant environment identity, root/cwd information, and a generation token.
3. Start one fresh pytest process with the same entrypoint semantics and an explicitly loaded PT bridge.
4. Let pytest perform normal configuration, imports, plugin loading, and collection.
5. Publish authoritative collected items and a plan revision from that **same live session**.
6. Execute with pytest's own run loop unless an already-authorized adapter owns scheduling.
7. Record final phase reports, finalizers, coverage/report artifacts, and pytest's exit status.
8. Verify completeness and generation status, reap the execution process, and emit the final result.

A separate `pt inspect --resolve` command MAY import and collect a project, but MUST warn that resolution executes Python code. It is not an implicit prerequisite to `pt run`.

3.3 Plans are progressive, not magically executable

Plan state is `requested -> indexed -> resolved -> running -> terminal`. An indexed plan may be useful to an editor but is not dispatchable. Resolution attaches runtime-backed identities, capability evidence, and a session token. Late plugin discoveries can revise or restrict capabilities before any affected work is dispatched.

The conservative path does not require the coordinator to pause pytest between collection and execution. It can observe the plan while pytest continues normally. A future scheduler requires an explicit handshake and conformance-tested integration point.

A serialized plan MUST NOT contain live fixture objects, callables, parameter objects, or a pickled pytest Session. A different worker may still need to import and collect the suite to reconstruct its own objects. Plan persistence alone does not remove that work.

3.4 Session state machine

```
created -> configuring -> collecting -> ready -> executing -> finalizing -> complete
                     \-> failed
any active state     -> cancelling -> terminated
any active state     -> infrastructure_failed
```

Each transition has a monotonic event sequence number. Terminal states are immutable. A crash during finalization is not `complete`. On coordinator restart, unfinished sessions become `abandoned` unless an authenticated, supported reattachment protocol proves otherwise. V1 does not reattach executing sessions.

4. Identity, plans, and persistent storage

4.1 Required identities

Identity	Required inputs and meaning
Request	Arguments, cwd, invocation profile, selectors, parallel policy, reporting requirements

Identity	Required inputs and meaning
Environment	Interpreter binary/build/ABI, executable path, relevant environment, installed distribution and plugin identities
Source generation	Content-derived workspace state, declared external inputs, symlink targets within the contract
Collection	Environment + request's collection-affecting options + source inputs + adapter identities
Plan	Collection identity + resolved inventory + constraints + selected execution policy
Session	Unique live process/session identifier; never reused as a cache key
Item occurrence	Session collection ordinal + exact node ID + worker/session token

Generation input manifests MUST distinguish source and declared data inputs from PT-owned metadata, test output, and temporary files. Store PT state outside the source snapshot or exclude it by an explicit output policy; never let a cache write masquerade as a user edit.

A lockfile is evidence of intended dependencies, not proof of installed files. Editable installs, `.pth` effects, namespace packages, local plugins, native extensions, and modified installed code require explicit identity treatment. Unknown external inputs make semantic caches non-authoritative.

Duplicate node IDs MUST NOT be silently deduplicated. Use occurrence identities and preserve multiplicity. Persist exact node IDs without assuming their bracket contents are serializable parameter values.

4.2 Fingerprints and validation

Use a versioned content-hash algorithm and canonical serialization with explicit field boundaries. Include schema and policy versions. Hash path bytes with a platform-aware encoding policy; preserve case-sensitive distinctions and report unsupported normalization collisions.

File watchers and stat data are invalidation accelerators, not sole correctness evidence. Same-size/same-mtime edits, atomic replacement, symlink retargeting, ignored files, and watcher overflow are required adverse tests. Full verification or a trusted snapshot boundary is needed before a cache is called validated. Benchmark the verification cost.

Environment-variable fingerprints MUST avoid exposing secrets. Store an allowlisted description and keyed digest where necessary; never place raw credentials in exported plans. A project requiring unknown environment inputs cannot claim a hermetic cache boundary.

4.3 Storage model

V1 uses a local transactional metadata store, with separate content-addressed blobs for large traces/artifacts. A relational implementation such as SQLite is acceptable; its selection is not an excuse to store unbounded logs in the hot index.

Logical tables: `environments`, `generations`, `requests`, `plans`, `item_occurrences`, `sessions`, `capability_decisions`, `duration_samples`, `dependency_edges`, `resource_leases`, and `artifact_manifests`.

All records carry schema/policy versions. Writes use transactions and atomic artifact publication. Corrupt or incompatible caches are discarded safely. Cache deletion MUST NOT delete project files. Garbage collection must respect active-session leases and bounded quotas. Secrets and large test stdout are not indexed by default.

5. Source indexing and semantic resolution

5.1 Static index

The Rust indexer records paths, content identities, candidate definitions, source ranges, literal markers where safely parsed, and provisional import relationships. It must tolerate syntax errors and unsupported syntax as index errors without suppressing pytest's authoritative error path.

Its responsibilities are editor navigation, change localization, and heuristic hints. The default full run MUST NOT exclude an unindexed file merely because it does not match an internal filename heuristic. Pytest configuration and custom collectors remain authoritative.

5.2 Resolution bridge

The bridge observes supported pytest hooks and reports collection outcomes and item occurrences. It must not serialize arbitrary parameter representations by calling expensive or side-effectful `repr` methods just for metadata. Human-readable parameter IDs supplied by pytest are sufficient.

The bridge is explicitly loaded only for PT runs. Packaging MUST NOT register an automatically enabled global plugin that slows down or changes a user's ordinary pytest baseline. Preserve `-p`, plugin autoload policy, `conftest.py` behavior, import mode, warning filters, and root resolution. Pytest has an ordered plugin-discovery process; changing that order requires separate compatibility evidence. [S08]

5.3 Collection reuse

Three states exist: `hint`, `validated_metadata`, and `reusable_runtime_preparation`. Most projects initially receive only hints. A validated metadata inventory still does not recreate live Python collection objects.

An adapter permitting collection reuse MUST declare its input domain, configuration/hook identities, invalidation rules, and state reconstruction mechanism. The absence of an observed file read is not evidence that no external input exists. Network, time, randomness, database contents, and generated files invalidate general claims of deterministic collection unless bounded by a stronger contract.

6. Capabilities and legal transformations

6.1 Capability record

Each capability has `decision`, `scope`, `issuer`, `issuer_version`, `evidence_digest`, `input_domain`, `reason`, and `expiry/invalidation_conditions`. The decision is `allow`, `deny`, or `unknown`; unknown is handled as deny for optimization.

Initial capabilities:

```
collection.metadata_reuse
collection.runtime_reconstruction
execution.partition
execution.reorder
execution.fork_framework_seed
execution.fork_project_snapshot
fixtures.replicate_scope
resources.reuse_service
selection.declared_inputs_complete
```

Capabilities are not inferred from an absence of obvious hooks. Project declarations are assertions with an explicit trust level, not automatic proof. Known adapters and user assertions must remain distinguishable in reports.

6.2 Composition and fallback

Compose capability scopes conservatively: every relevant plugin, hook, fixture policy, and project constraint must permit a transformation. Conflicting assertions disable the transformation. Plugin upgrades invalidate the corresponding evidence.

Unknown global collection/session hooks force the relevant session-level fallback. A module-level fallback is legal only when its independence has been established. Runtime detection after effects begin stops or continues conservatively in the existing session; it never transparently restarts the test under another backend.

A user-requested optimization can be `required` or `preferred`. A denied required capability is a configuration failure before execution; a denied preferred capability uses the conservative route and records the reason.

7. Process supervision and warm preparation

7.1 Disposable execution sessions

Every completed or aborted execution session is destroyed. Freshness is per run/session, not an attempt to reset `sys.modules`. This avoids relying on a partial list of state to undo. Pytest documents import-cache limitations when invoking `pytest.main()` repeatedly in one process. [S09]

The supervisor owns child lifetime, cancellation, pipe drainage, timeout escalation, and process-tree cleanup. Native process groups/job-control primitives are used per platform. V1 is a process-lifecycle controller, not a security sandbox: hostile tests may escape ordinary ownership boundaries unless stronger isolation is separately configured.

Hard timeout records the affected phase, terminates the owned worker boundary, marks unfinished work interrupted, and reports which teardown guarantees were lost. It never reports a timeout as an ordinary assertion failure followed by a successful automatic retry.

7.2 Framework seed experiment

An optional, dedicated, single-threaded Python seed may import an audited framework baseline and create disposable children. It must not import project code, load arbitrary project plugins, execute tests, or own application sockets. Forking occurs in that Python seed, never inside the multithreaded Rust coordinator.

Python's documented fork restrictions and platform differences make this an explicit platform-specific capability. Spawn/fresh execution remains the fallback. Even a framework preload changes import timing and can affect instrumentation or import-side effects; it requires differential tests rather than being assumed universally harmless. [S10]

Measure seed creation, first child, steady children, invalidation, idle memory, copy-on-write breakage, child cleanup, and interpreter/plugin upgrades. A seed that is already dirty or incompatible is destroyed before use.

7.3 Project snapshots

Project snapshots are research-only until their input and fork-safety contracts are established. No open database connection, thread pool, browser process, GPU context, or arbitrary event loop is assumed cloneable. A snapshot adapter must define resource reinitialization and demonstrate that collection-side effects are not duplicated or omitted relative to its stated reference model.

A project snapshot may legitimately use a different execution contract from direct fresh pytest. Report that distinction; do not hide it inside a generic “warm run” label.

8. Scheduling, fixtures, and managed resources

8.1 Scheduling objective

Minimize measured completion or failure-feedback latency subject to legal ordering, fixture-lifetime constraints, CPU/memory limits, and resource limits. The cost model includes startup, collection, imports, setup, call, teardown, and reporting. Historical duration is advisory and partitioned by environment identity.

The initial parallel backend delegates to pinned xdist. xdist workers normally collect independently and verify matching collection inventories; replacing this behavior requires evidence that reconstruction and hook effects remain valid. [S04]

A later scheduling adapter may use duration-aware grouping and work stealing through a supported integration point. It must not pretend a fixture scope is merely a performance hint. Parametrized fixtures, dynamic `request.getfixturevalue`, and session-aware fixtures can constrain grouping. Unknown dependencies retain the existing backend's behavior.

8.2 Resource model

A managed resource is an external service lease, not a cached Python fixture instance. A worker obtains an endpoint/capability and creates its own connection and isolation namespace. Lease state is `requested -> acquired -> active -> releasing -> released/expired`.

Requirements include capacity limits, exclusive modes, fair admission, cancellation-aware release, worker-death recovery, and a heartbeat/expiry policy. Service reuse across runs is opt-in and must specify data reset and contamination checks. Shared-service setup savings are reported as such, not as faster test dispatch.

Acquire multiple resources in a deterministic global order or through an atomic reservation mechanism. Detect impossible requests and avoid deadlock. Leases cannot bypass a test's declared session or database isolation semantics.

8.3 Parallel failures

Preserve original attempts. Retries are disabled by PT by default; a configured retry plugin belongs to the declared backend contract. Parallel fail-fast may have in-flight work; report both the first failure time and final shutdown time, including extra work completed after the stop decision.

9. Incremental selection and source-generation integrity

9.1 Dependency graph

Record edges from test occurrences to executed code, fixtures, configuration, and declared data inputs, with provenance and confidence. Include collection dependencies separately from execution dependencies. A session fixture used by many tests must not be attributed only to the first test that triggered setup.

Dynamic observations are incomplete observations. Threads, subprocesses, native extensions, generated code, reflection, and external services may hide dependencies. Unsupported attribution broadens selection or disables it; it must not silently produce narrower confidence.

Selection has two modes: **observational** (heuristic, explicitly scoped) and **declared-input** (bounded by a stronger project/adapter contract). Neither is advertised as a universal proof of full-suite success.

9.2 Selection rules

Always include new/changed tests, previously failed tests, tests with stale/missing dependency evidence, and tests invalidated by relevant fixtures/configuration. Unknown broad inputs cause a full-scope run. Renames and deletions require inventory reconciliation; an empty selection is not a successful full run.

V1 selection takes place after authoritative collection inside the normal session. Avoiding collection itself is a separate capability. Changed-file indexing is therefore useful even before collection-free incremental execution exists, but benchmark its actual latency rather than assuming collection disappears.

9.3 Generations and concurrent edits

Every result names a generation and an observation model. Strict mode executes an immutable snapshot covering declared inputs. Snapshot creation and path mapping costs belong in end-to-end timing. Arbitrary absolute paths and undeclared external data can make strict mode unsupported.

Live-tree mode marks results `superseded` if a relevant change is observed during the run and schedules a new request. A watcher plus end-of-run hashes cannot prove that a transient edit-and-revert was never read. Accordingly, live-tree results MUST NOT claim atomic-snapshot assurance. Watcher overflow or uncertain integrity forces rescan and an uncertainty status, not green.

A late result for generation G may appear in history after G+1 exists, but cannot update G+1's current status. Cancellation is policy-controlled; cancelling a running test can have external effects, which must remain visible.

10. Event protocol and reporting

10.1 Transport

Use a versioned, length-delimited message protocol over a dedicated local channel. V1 can use UTF-8 JSON payloads for inspectability. Test stdout/stderr never carries control messages. Set frame-size limits, stream large output separately, and reject malformed frames without interpreting arbitrary Python objects.

Messages contain protocol version, request/run/generation/session IDs, sequence number, event kind, and a typed payload. Monotonic times are valid within their recorded clock domain; cross-process timestamps are not subtracted blindly. The external benchmark harness owns authoritative end-to-end timing.

Guarantee ordered, loss-detectable delivery within each producer. Duplicate delivery is handled by event identity. This is not a promise of exactly-once execution: a process may perform an external action immediately before it dies. No automatic retry is justified by a missing acknowledgement.

10.2 Required events and outcome model

Events include request acceptance, environment resolution, collection reports, item inventory, capability decisions, phase reports, warnings, output references, resource events, worker death, cancellation, artifact commit, and session termination.

Store setup/call/teardown reports separately, including skips, xfail/xpass reasons, strictness, and rerun attempt identity. Maintain collection errors, internal errors, and infrastructure failures at session level. Subtests and custom collectors require versioned extension records, not flattening into one pass count.

A final summary contains requested scope, collected multiplicity, deselected count, selected count, executed occurrences, outcomes by phase, incomplete work, generation status, and backend exit status. Missing reports cannot be inferred as passes.

10.3 Output and completion

Default user-facing pytest formatting should remain available. Structured reporting is additive. Capture mode, color, TTY status, warning policy, traceback verbosity, and output destination are part of a benchmark scenario.

A run is complete only when required reports and coverage artifacts are committed, required execution processes are reaped, and required lease cleanup is complete. An optional warm coordinator/seed may remain; its live resources are reported. Best-effort indexing can finish later, but any work needed before the next valid request belongs in the replay endpoint and cannot be hidden as free background work.

11. User interface, packaging, and failure policy

11.1 Proposed commands

```
pt run                                    # Full conservative run
pt run -- tests/test_orders.py -q          # Pytest arguments after --
pt run --parallel-backend xdist --jobs 4 -- --dist=loadscope
pt watch --selection affected             # Explicit affected scope
pt inspect --resolve                      # Explicit import/collection operation
pt explain --run <run-id>                  # Decisions and provenance
pt doctor                                 # Identity, version, cache, backend checks
pt cache clear                            # PT-owned metadata only
```

`pt` may be an alias for `pt run`. Reserve PT options before `--`; forward trailing arguments exactly. Do not reinterpret or quietly ignore unknown pytest/plugin options. Invocation profiles distinguish the installed pytest console entrypoint from `python -m pytest`, since pytest documents a `sys.path` difference. [S09]

Configuration is optional under `[tool.pt]`. Existing pytest configuration is preserved. `--no-daemon`, `--no-cache`, and `--no-preload` diagnostic switches are required, but disabling features does not disable correctness checks.

11.2 Exit and scope semantics

For a completed real-pytest session, preserve its exit status. Pytest documents distinct codes for test failure, interruption, internal error, usage error, and no collected tests. [S11]

PT-originated configuration failures use the documented usage-error mapping; PT infrastructure failures use the documented internal-error mapping, with richer machine-readable cause fields. Signal termination preserves platform-appropriate information. A zero exit from an affected run means the selected work succeeded, not that the full suite was verified. Uncertain or superseded generation integrity does not emit a current successful result.

11.3 Installation and trust

Ship a Rust binary plus a small interpreter-compatible bridge distribution. Never install a replacement package named `pytest`. Do not silently install, upgrade, or mutate dependencies during execution. Validate the bridge/backend version pair before starting project work.

Local IPC is user-scoped with appropriate permissions and a per-session authentication token where needed. Do not execute cache-provided pickle objects. Hashes verify identity, not trustworthiness. Tests remain executable code with the user's authority unless separately sandboxed.

12. Benchmark charter: what a claim is allowed to mean

12.1 Three claim families, never one blended score

Family	Question	Required comparison	Prohibited interpretation
A: Equal-work execution	Is the same requested work completed faster?	Same inventory/selection, contract, resources, coverage, reporting, and input state	Comparing a selected subset with a full suite
B: Prepared-session amortization	Does persistent preparation help over a realistic sequence?	Include preparation, invalidations, idle cost, and an explicit number of interactions	Quoting only the best already-warm invocation
C: Incremental feedback	Does affected selection deliver useful, sufficiently reliable feedback sooner?	Replay edits; report selection size and failure-detection audits against full runs	Calling fewer executed tests a faster full runner

Every published result must belong to exactly one primary family. Secondary breakdowns are allowed, but aggregation across families is not.

Primary product endpoint: wall-clock latency from request submission to a complete, generation-valid result under the declared scope. For watch mode, begin at the harness-confirmed edit commit boundary and include notification delay, debounce, selection, execution, and required reporting.

Secondary endpoints: time to first actionable failure, CPU time, total live memory, repeated-session cost, and correctness/failure-detection evidence. Throughput is useful, but cannot substitute for interactive latency.

12.2 Preregister the comparison

Before confirmatory timing, commit an immutable campaign manifest containing the hypothesis, candidate/baseline artifacts, scenario list, corpus eligibility, resource allocation, exact commands, cache states, warmup/reset rules, run order seed, sample size, endpoint definitions, exclusion criteria, analysis method, and publication gates.

Exploration is allowed and encouraged. Exploratory results choose optimizations and estimate variance; they cannot also serve as the untouched confirmatory test. After tuning, freeze the candidate commit and use held-out repositories or held-out edits plus a new campaign.

A protocol change after viewing results creates a new campaign ID. Preserve the original campaign and explain the change. Do not rewrite unsuccessful evidence out of the record.

13. Baselines and fair configuration

13.1 Required baseline set

ID	Baseline	Role
B0	Pinned direct pytest, serial	Semantic reference and primary serial performance baseline
B1	Pinned pytest + xdist, fixed worker count	Parallel baseline, same CPU/memory budget
B2	Tuned permitted xdist policy	Strong parallel baseline chosen on training workloads, not the test set

ID	Baseline	Role
B3	Pinned pytest + testmon	Incremental-workflow comparator where its supported contract applies
B4	PT conservative backend, optimizations off	Attribution baseline for PT's own features, not a replacement for B0/B1
B5	pytest-rs, rpytest, rupt at recorded revisions	Optional competitive experiments after compatibility qualification

Do not make a competitor's unsupported test cases disappear. Publish eligible count, unsupported count, wrong-result count, crash count, and timeout count. A common-compatible-subset comparison is allowed only when named explicitly and accompanied by the excluded portion of the original corpus.

Competitors with incompatible installed-package requirements use isolated environments with matched application dependencies. Record every remaining difference. Never place conflicting runners in one environment just to simplify command invocation.

13.2 Fairness checklist

Match interpreter binary/build, project revision, installed application dependencies, plugins, plugin loading policy, warning filters, test selectors, environment inputs, parallelism budget, native-library thread counts, capture mode, coverage configuration, reporting destinations, and service state.

A baseline can legitimately have fewer PT-specific packages. The important requirement is that PT-specific packaging must not autoload into or otherwise slow the baseline. Do not add PT's production bridge to baseline timings merely to make PT look less expensive. If a common measurement probe is needed, measure its cost and report the instrumented result separately.

Use the same optimized, supported configuration for both sides of a controlled comparison. Disabling plugin autoload only for PT, running pytest with expensive coverage contexts only on one side, or comparing release Rust against a debug baseline invalidates the cell. A separately labelled “best supported configuration” comparison may differ in configuration, but must describe those differences and cannot serve as an equal-configuration attribution result.

Build Rust artifacts in the release profile intended for distribution. Record compiler, target, optimization flags, allocator, LTO/PGO configuration, and binary digest. `target-cpu=native` may be a separate local-build profile, not an undisclosed advantage over portable distribution artifacts.

13.3 Parallel and invocation controls

Sweep fixed worker counts such as 1, 2, 4, and a recorded host-appropriate maximum. `-n auto` is not a reproducible fixed-resource comparison unless the resolved count is captured and equalized. Include the coordinator, seed, resource services, and library threads within the declared budget. Avoid counting only workers while giving PT extra CPUs.

Select xdist scheduling policies using a training set and a declared tuning budget. Report the default policy and the tuned policy; do not select the fastest policy separately after seeing every held-out result.

For engine-only timings, invoke resolved binaries directly with argv arrays and no shell. Dependency resolution, environment creation, downloads, and Rust compilation occur outside engine timing but are recorded. A separate developer-workflow experiment may include `uv run` or environment setup symmetrically and must name that larger boundary.

14. Benchmark corpus

14.1 Three corpus tiers

Mechanism probes isolate startup, collection, setup, execution, capture, or reporting. They explain causes but are not the headline product score.

Representative repositories exercise real dependency/plugin combinations. Freeze revisions and installation artifacts. Target at least twelve repositories spanning small libraries, import-heavy applications, web/database projects, async projects, numerical/native-extension projects, plugin-heavy suites, and projects with custom collection. If fewer qualify, narrow the published claim rather than pretending coverage is broad.

Adversarial conformance projects intentionally exercise dangerous boundaries. They may be slow or pathological; their purpose is detecting wrong behavior, not inflating the performance average.

WP02 establishes an inventory and separates training, confirmatory, and never-tuned holdout sets. Proposed corpus candidates are selected by documented workload properties, not because PT already performs well on them. Pin real revisions during implementation; this spec does not invent commit hashes or pretend repositories have been validated.

14.2 Required mechanism probes

Probe	What it isolates	Necessary controls
Empty suite and one no-op test	CLI/interpreter/framework startup	Preserve no-tests exit behavior; no fake work counts
Many tiny tests	Per-item framework and event overhead	Verify every item and phase actually executes
Import-heavy collection	Import and collection preparation	Count import side effects; cold/warm pyc split
Dynamic parametrization/ custom collector	Runtime inventory	Exact identities and multiplicity
Expensive shared fixtures	Scheduling/setup replication	Record setup and teardown multiplicities
Skewed test durations	Scheduling quality	Fixed resource budget and declared reordering
Python CPU-bound tests	Application-dominated floor	Expect limited orchestration benefit
Native-library parallel tests	Thread oversubscription	Pin/record BLAS/OpenMP thread policy
I/O/service-bound tests	External latency and resource reuse	Local controlled service; reset data
Output-heavy failures	Capture, rendering, backpressure	Same verbosity, destination, and bytes
Coverage workloads	Measurement/merge cost	Same line/branch/context semantics
Crash, hang, cancellation	Failure-path correctness and latency	Include process-tree cleanup

At least one negative-control workload must have almost no removable framework cost. A large claimed gain there triggers investigation of changed work, resource allocation, or measurement boundaries.

14.3 Adversarial correctness inventory

Required cases include autouse fixtures; fixture scope/override/parametrization; dynamic fixture lookup; yield/ finalizer exceptions; setup skip/failure; teardown error after passing call; strict xpass; collection error; duplicate node IDs; unicode paths/IDs; doctests; unittest integration; custom collectors; hook wrappers and ordering;

warnings-as-errors; stdout/stderr file-descriptor capture; async plugins; property-based test configuration; subprocess creation; import-time database writes; module-global mutation; repeated invocations; a plugin loaded dynamically during collection; and a session hook that examines all selected items.

Change-integrity cases include same-size/same-mtime edits, edit-and-revert, file deletion/rename, symlink retargeting, changed data outside the repository, environment changes, editable dependency edits, corrupted caches, watcher overflow, dependency upgrades, and stale bytecode. Import tests must detect same-second source changes that a naïve timestamp-based setup could mishandle. The harness must verify intended source was actually loaded; purging bytecode is allowed only as an explicitly symmetric scenario policy.

Fault injection covers lost/duplicate/truncated events, worker death before acknowledgement, coordinator death, disk-full artifact writes, broken pipes, orphaned descendants, OOM termination, invalid protocol versions, and interrupted coverage merging.

15. Cache and lifecycle state matrix

15.1 “Cold” is a vector, not a label

Each sample records at least these axes:

```
coordinator: absent | running
framework_seed: absent | validated | invalidated
pt_metadata: absent | populated | invalidated
python_bytecode: absent | valid | controlled-invalid
filesystem_cache: uncontrolled | warm-procedure | cold-procedure
application_service: absent | fresh | reusable-reset
dependency_observations: absent | trained | invalidated
source: unchanged | edited | environment-changed
```

A deleted PT cache does not make the OS page cache cold. A fresh process does not imply cold filesystem or bytecode caches. A reboot does not necessarily establish a particular device-cache state. Use precise labels and attach reset/probe evidence.

15.2 Mandatory scenarios

ID	Scenario	What the timer includes
S0	First usable run, no PT state	Coordinator/seed creation, identity, execution, reporting, required cleanup
S1	Fresh execution, warm filesystem/valid pyc	All process startup; no reused execution interpreter
S2	Warm coordinator, unchanged full run	Request, validation, fresh session, full work, finalization
S3	Targeted test after application edit	Invalidation, validation, selected explicit work, execution
S4	Test/parametrization edit	Correct new inventory and execution
S5	Conftest/config/data edit	Required broad invalidation and rebuild
S6	Dependency/interpreter/plugin change	Environment re-identification and all invalidated preparation
S7	Affected watch sequence	Edit-to-valid-result, dependency training amortization, selection audit

ID	Scenario	What the timer includes
S8	Failure/hang/cancel	Actionable report plus actual owned-process cleanup
S9	Explicit OS-cold experiment	Approved cold-state procedure, documented separately

S9 is optional when reliable cache control is unavailable. Never clear global OS caches on shared machines. Any privileged system tuning must be intentional, recorded, and restored outside production environments. pyperf's documentation emphasizes system tuning and inspection of unstable results rather than assuming repeated runs are automatically reliable. [S13, S14]

15.3 Reset scripts are executable contracts

Every arm has a reset procedure that restores equivalent preconditions, not necessarily identical files with identical names. PT may have a daemon while direct pytest does not; that is the feature being evaluated. Its cost still belongs in the applicable scenario.

Reset service/database state, generated files, benchmark-specific caches, failure-history stores, and dependency observations as declared. Use separate arm worktrees and cache roots. Rotate or symmetrically prepare them to prevent first-arm page-cache effects. For warm scenarios, run a fixed, preregistered warming procedure for both arms. For first-run scenarios, do not warm the timed artifact while calling it cold.

Preparation excluded from one timed invocation must either establish the scenario's stated initial state or be included in its amortized sequence. It cannot perform work the product would otherwise have to do for free.

16. Harness design and experimental procedure

16.1 Independent outer harness

Implement `pt-bench` as a separate Python package or executable that can benchmark PT, direct pytest, and competitors without using PT's own timer as the authority. Its control code must not be imported into measured test workers unless a specific instrumentation experiment requires it.

The harness owns monotonic wall timing, process invocation, timeouts, experiment order, state preparation, resource collection, result validation, and immutable raw artifact publication. It launches commands with argv arrays and records exact cwd/environment differences. Do not scrape only the final “X passed” line and call that proof of equal work.

A successful sample requires a valid terminal outcome, required artifacts, expected test-work evidence, valid generation, and reset-policy compliance. Failure-oriented scenarios intentionally expect nonzero outcomes; their expected error class is part of the manifest.

16.2 Separate measurement layers

Layer 1: unmodified product timing. Time the shipping behavior, including PT's production bridge. The baseline has no artificial PT overhead.

Layer 2: correctness-instrumented runs. Add a common minimal observer or test-owned sentinels to establish detailed semantic equivalence. Compare instrumented and uninstrumented behavior to identify observer effects. Synthetic/adversarial suites can emit their own deterministic sentinels without invasive introspection.

Layer 3: diagnostic profiling. Collect detailed import, phase, allocator, CPU, or system traces. These explain bottlenecks but are not headline timing samples unless the profiler is part of the shipping product.

Correctness qualification before/after timing plus per-sample lightweight inventory/accounting guards is required. If a workload cannot be validated without expensive instrumentation, report that limitation and keep its evidence class explicit. An instrumented equivalence result is not proof that an uninstrumented, nondeterministic run did exactly the same work.

16.3 Paired, randomized blocks

The default comparison unit is a **paired block** containing one baseline and one candidate observation, each restored to its declared starting state. Randomize AB versus BA with a committed seed and balance order within each batch. With more than two arms, use balanced randomized permutations; never run all baselines in the morning and all candidates after the machine warms up.

For a warm-session replay, one observation is an entire newly bootstrapped session containing a fixed edit/run sequence. Multiple interactions within that session are correlated; they are not counted as independent replicates. Pair complete sessions with identical edits, input seeds, and external state.

Do not run competing arms concurrently on the same resource allocation. Record thermal/power state, CPU governor, CPU affinity, SMT policy, NUMA placement, container limits, background activity, memory pressure, storage, and virtualized/shared-host status. Default controlled native-library thread limits should be identical; a realistic unconstrained profile can be reported separately. ASV documents both time-scale effects and native-library threading as relevant measurement concerns. [S15]

16.4 Campaign procedure

1. Freeze the manifest and candidate/baseline artifacts; record its digest.
2. Materialize pinned environments and independent worktrees; record wheel/file hashes and machine metadata.
3. Run the correctness qualifier and confirm expected inventory/outcomes.
4. Run a separate pilot to estimate variability and choose a fixed confirmatory sample count.
5. Generate balanced randomized block order; prepare and verify each arm's declared state.
6. Measure the complete invocation or session replay; collect lightweight validity evidence and resource totals.
7. Store every sample, including warmups, crashes, timeouts, mismatches, and excluded infrastructure failures.
8. Repeat the correctness qualifier and check source/environment identities remained fixed.
9. Analyze the frozen sample set; publish intervals, absolute costs, denominators, and all disqualifications.
10. Reproduce the release-critical result independently on another host or machine session before making a broad claim.

The workflow is fail-closed: unresolved artifact hashes, unknown expected outcomes, missing reset scripts, or unregistered exclusions block confirmatory status. Exploratory mode can proceed, but its report is visibly labelled exploratory.

16.5 Sample sizes and stopping

Proposed confirmatory default: at least **30 independently reset paired blocks per primary short/medium scenario**, distributed across at least three separated time windows. Long scenarios use pilot-based fixed counts with a documented minimum of **12 independent paired blocks**; lower counts are exploratory unless a preregistered alternative design justifies them.

These minima are policy starting points, not guarantees of adequate precision. Select final N from pilot variance and the smallest practically important effect; freeze N before confirmatory results are inspected. If the interval remains too wide, report inconclusive and run a newly registered extension, not “repeat until significant.”

Bootstrap whole independent blocks. When many blocks share state that may induce correlation, aggregate or resample at that higher level. Three time windows do not provide enough independent clusters for a strong cluster-bootstrap claim by themselves. Collect additional independent sessions or explicitly limit the uncertainty statement to the observed host/sessions. Time-order plots and residual dependence checks are required; do not silently assume independence.

Tail-latency claims require enough observations of the claimed tail. Proposed policy: at least 200 valid observations for an exploratory p95 estimate with an uncertainty interval; the number of independent outer sessions must also be reported. A p95 calculated from twelve runs is not release-grade tail evidence. Even 200 observations can be inadequate for a narrow interval.

16.6 Outliers, failures, and censoring

Keep slow samples by default. Thermal throttling, GC, occasional expensive setup, and PT-induced contention are real costs unless the campaign explicitly studies a different controlled regime.

An external infrastructure event may invalidate a paired block only under a preregistered rule supported by evidence independent of its favorable/unfavorable timing. Retain the original records and invalidate the entire pair, not just the slow arm. Publish exclusion counts and reasons.

Product crashes, hangs, wrong inventories, and memory exhaustion are product outcomes, not removable statistical outliers. A timeout is censored duration plus a reliability failure; do not replace it with the timeout threshold and include it as a successful runtime. Failures disqualify the cell from a successful-speedup claim while remaining in the coverage/reliability denominator.

17. Metrics and analysis

17.1 Required raw metrics

Metric	Definition
End-to-end wall time	External request start to complete valid result/required cleanup
First actionable failure	Same start to first user-visible failure with sufficient diagnostic information
Worker/session startup	Request to relevant process ready, with clock-domain provenance
Collection and phase times	Diagnostic backend observations; not additive across overlapping workers
CPU time	User + system usage for the full measured execution boundary, including persistent processes' deltas
Peak charged memory	Platform-defined whole-boundary peak; exact metric and scope recorded
Idle residency	Coordinator, seed, and reusable services between requests
Executed work	Exact selected occurrences and phase/outcome accounting
Fixture/import effects	Instrumented multiplicities and side-effect sentinels where available
Coverage	Measured file/line/arc/context sets and merge status where required
Integrity	Generation, identity, capability decisions, fallbacks, invalidations

Measure first actionable failure both with fixed order and with explicitly permitted prioritization. A reordered failure discovered earlier is a scheduling result, not necessarily faster execution of a fixed order. Right-censor scenarios with no failure and report their absence; do not invent a failure latency.

17.2 Memory accounting

On Linux, a dedicated cgroup can provide boundary-wide charged-memory and CPU accounting. `memory.peak` covers the cgroup and descendants; charged memory is not the same quantity as process RSS and may include cache charges. Record the exact metric, accounting scope, kernel capabilities, and reset behavior. [S16]

A warm daemon and seed must be included in the boundary from creation. Account for precharged/shared pages and ensure a pre-existing process outside the cgroup does not make PT look artificially cheap. Use a dedicated group per independent session where feasible. For per-request warm metrics, retain whole-session peak and sampled current usage if precise peak reset semantics cannot be established.

Do not sum each process's historical maximum RSS and label that simultaneous peak memory. A sampled sum of current RSS is a different metric and may double-count shared pages. PSS-based samples, charged-memory peaks, and platform job/accounting metrics must be labelled separately. Supplement memory latency runs with dedicated measurement passes when high-frequency sampling perturbs timing.

Report CPU-seconds and memory cost alongside wall-time gains. Faster by using twice the resources is a tradeoff, not free efficiency.

17.3 Effect size and confidence intervals

For a comparable paired observation b, define:

```
d_b = log(T_baseline,b / T_candidate,b)
G   = exp(mean(d_b))
```

G greater than 1 indicates a speedup; G below 1 indicates a regression. Report both G and absolute latency for each arm, including the paired absolute difference. State whether T is a single invocation or the total measured session replay. Do not mix them.

Use a preregistered paired bootstrap of independent units, with a fixed analysis seed and at least 10,000 resamples, for an uncertainty interval on the chosen estimator. Where session/host dependence exists, use appropriate aggregation or hierarchical/block resampling and report the effective independent-unit count. Small numbers of clusters require caution; a narrow computed interval is not automatically credible.

The primary estimate is not a ratio of two independently selected minima. Medians and p95s are useful descriptive endpoints but require their own interval method. The arithmetic mean of speedup ratios is not the geometric-mean estimator above.

For a proposed meaningful gain threshold g (for example, 5%), claim a meaningful gain only when the relevant confidence bound exceeds 1 + g and the absolute savings clear a preregistered practical threshold. If the interval crosses 1, say uncertain; if it excludes 1 but not 1 + g, say detectable but below the chosen practical gate. Do not translate “not statistically significant” into “equivalent.”

17.4 Aggregation and multiplicity

Publish every repository/scenario cell. Primary aggregates use fixed preregistered weights, with equal repository weight within a declared scenario family; thousands of synthetic test cases must not outweigh a small number of real projects. Report geometric-mean ratios and a separately labelled absolute total-time comparison for a declared workload mix.

No single grand mean across cold, warm, selected-subset, and full-suite scenarios. No switching weights after results are known. Missing, failing, and unsupported cells remain visible and reduce the claim's eligible coverage. A common-compatible aggregate is not a whole-corpus aggregate.

Preregister a small set of primary endpoints. Secondary cell intervals are exploratory unless adjusted for multiple comparisons. For simultaneous per-cell claims, use a declared family-wise procedure such as Holm-adjusted tests or a simultaneous bootstrap interval procedure. An aggregate gain does not excuse a correctness failure or an unacceptable individual regression.

17.5 Attribution and sanity checks

For unchanged application/fixture time fraction f, eliminating all other work gives an upper bound of approximately 1/f on serial equal-work speedup. Use measured phase estimates carefully because overlap and instrumentation affect attribution. A surprising violation is a prompt to inspect changed work/resources, not evidence that arithmetic has been defeated.

Perform feature ablations on the same workload: conservative PT; plus indexing; plus metadata reuse; plus framework seed; plus scheduling; plus dependency collection; plus affected selection. Do not use affected selection as evidence of equal-work execution gain. Test selected feature interactions; incremental improvements need not add linearly.

18. Coverage and diagnostic equivalence

Coverage experiments match coverage.py version, actual measurement core, Python version, source/include/omit configuration, line versus branch measurement, contexts, plugin usage, subprocess behavior, and final artifact requirements. Coverage.py exposes multiple cores with version-dependent feature support; requesting a core is not proof that the run actually used it. Capture fallback warnings and the effective configuration. [S17]

Compare measured file identities and line/arc sets, not only a rounded percentage. If the test contract legitimately changes order or concurrency and yields nondeterministic paths, record a coverage uncertainty class rather than silently accepting differences. Verify subprocess coverage and required merge behavior explicitly.

The full endpoint includes required merge and requested report generation on both sides. A separate execution-only measurement may exclude reporting only when named and controlled symmetrically. Dynamic dependency instrumentation is also charged to PT's ordinary execution cost when enabled; it is not free just because it helps a later selected run.

Keep established coverage integration in V1. A future replacement must earn its own semantic and performance qualification rather than inherit the test runner's claim.

19. Incremental-selection evaluation

19.1 Replay corpus

Build two complementary sets: real historical edits with known parent/child revisions and synthetic controlled changes with a known relevance model. Include passing edits, failure-inducing edits, new tests, changed parametrization, fixture/config/data changes, import-graph changes, deleted tests, environment changes, and broad refactors.

Historical bug fixes run in the correct direction for the experiment. A test added after a bug is fixed cannot be assumed to have existed during the original developer loop. Record exactly which code and test versions are present at every replay step.

For each replay, train dependency observations using the permitted initial full run, apply a committed sequence of edits, and measure selected execution. Independently run a clean full oracle on the same revision and environment, with external state reset. The oracle's audit cost is not part of the hypothetical user's selected-run latency, but is included in the campaign's cost record; actual product-required audits belong in product timing.

19.2 Selection metrics

For a revision with stable oracle failure set F and selected inventory S:

```
failure recall = |F intersect S| / |F|       when |F| > 0
missed failures = F minus S
selection fraction = |S| / |full inventory|
```

Also compare the observed selected outcomes with the full oracle's outcomes. Selecting a failing test is insufficient if PT changes it into a pass. Failures in collection, session setup, or infrastructure are separate session-level detection events, not squeezed into a test-level denominator.

Failure recall measures known failing tests, not all semantically affected tests. A passing full oracle provides no positive evidence that every affected passing test was selected. Selection precision/affected-test recall require a known dependency or mutation ground truth and cannot be inferred merely from pass counts.

Report per-edit and aggregate missed-failure counts, eligible failure-inducing edits, unsupported/uncertain edits, full-fallback rate, selection fraction, edit-to-result latency, and training/instrumentation cost. Recheck flaky oracle failures; classify instability rather than silently dropping inconvenient cases.

19.3 Proposed release safety gate

No false current-generation green, no missed known stable failure on the mandatory supported replay corpus, and correct full fallback on unknown broad changes. This is a zero-observed-miss gate, not a claim that the miss probability is zero.

As a transparent scale check, if N independent, representative failure-inducing edits have zero misses, the one-sided 95% binomial upper bound on miss probability is `1 - 0.05**(1/N)`, approximately `3/N` for large N. At N = 300 it is approximately 1%. Correlated mutations, multiple tests from one edit, and curated easy cases do not provide 300 independent representative trials. Report the exact assumption limits.

Collect at least 300 diverse failure-inducing edits as an initial evidence target, distributed across repositories and mechanism classes. If they are highly clustered, report cluster-aware uncertainty and do not quote the simple bound as a population guarantee. Full CI remains the default, even after this gate passes.

19.4 Amortization and break-even

For N interactions, compare the actual total costs:

```
C_PT(N) = preparation + dependency_training + sum(valid_interaction_costs)
          + invalidation_rebuilds + required_shutdown
C_B(N)  = baseline_preparation + sum(baseline_interaction_costs)
          + baseline_required_shutdown
```

Avoid double-counting any cost already inside an interaction timer. Include background work that competes for resources or must complete before the next request. Report N = 1, 5, 20, and 100 when the workload supports it, plus the observed break-even interaction count. Include edit/invalidation mixes representative of the claimed workflow rather than only unchanged no-op reruns.

Idle residency and optional energy cost are separate dimensions, not automatically converted to seconds. A feature may improve latency and still be rejected for excessive persistent memory or CPU use.

20. Publication contract and benchmark self-tests

20.1 Minimum result card

Every claim includes candidate/baseline revisions and binary digests; Python/pytest/plugin versions; repository revision; exact commands; scenario/cache vector; hardware/resource budget; collection and execution denominators; semantic contract; sample/independent-session counts; elapsed time and uncertainty; CPU/memory; coverage policy; fallbacks; exclusions; and a reproducibility bundle digest.

The report displays correctness qualification and unsupported coverage before speedup. Label results **exploratory**, **qualified**, or **release-confirmatory**. A missing field prevents the higher classification.

An acceptable headline has the form: “On this fixed corpus and profile, the equal-work geometric-mean speedup was X with interval Y; Z of K repositories qualified, with these regressions and resource costs.” No numerical X/Y/Z values exist until the protocol has been run.

20.2 Red-team the harness

Before trusting it, deliberately create broken candidates that skip tests, skip teardown, suppress warnings, omit branch coverage, reuse stale modules, report cached passes, leave workers alive, and begin work before the timer. The harness must reject or correctly classify each advantage.

Inject a known sleep into startup, collection, teardown, coverage merge, and shutdown. The correct endpoint must increase accordingly. Inject extra background CPU and persistent memory; accounting must expose them. Swap baseline/candidate labels to verify ratio direction. Replay the same raw data through the analyzer twice and require identical outputs with the fixed seed.

Construct autocorrelated synthetic sample streams and ensure the analysis does not claim hundreds of independent observations from one warm daemon. Exercise order imbalance, missing records, NaN/negative durations, zero denominators, duplicate samples, and edited manifests. These are harness acceptance tests, not optional hygiene.

20.3 Tool roles

Use pyperf for component-level Python measurements and result inspection, and ASV-style historical tracking where useful. General command timers such as hyperfine are useful for exploration and smoke checks, but do not by themselves implement PT's semantic oracle, stateful replay, whole-process accounting, or experiment governance. Pin their versions and actual invocation semantics rather than copying flags from memory. [S13–S15, S18]

21. Work packages

Each package has one accountable role, explicit dependencies, deliverables, and acceptance evidence. Roles can be held by the same person, but benchmark review should not depend solely on the author of the optimization being evaluated. Package completion means its evidence is committed and reproducible, not merely that code has merged.

WP00 — Contract, support matrix, and architectural decisions

Owner: technical lead. **Depends on:** none. **Priority:** P0.

Freeze invocation profiles, initial interpreter/OS/backend versions, C01–C12, protocol v0, benchmark claim taxonomy, and unsupported-case policy. Write architectural decision records for subprocess versus embedding, real pytest versus replacement, serial default, explicit parallel semantics, and no automatic replay after effects begin. Define the exact initial full-run completion boundary and supported stdout/exit behavior.

Deliverables: `docs/contracts.md`, `support-matrix.toml`, protocol draft, threat model, ADR001–ADR006, and benchmark charter.

Acceptance: every optimization has a named permission and reference contract; every exit path has a typed outcome; no requirement implies arbitrary collection can be serialized or arbitrary state can be reset. Review using the adverse examples in section 14.3.

Failure/stop condition: unresolved disagreement over what “compatible” means blocks implementation of performance transformations. Resolve the contract before comparing timings.

WP01 — Independent benchmark harness and raw evidence format

Owner: performance engineer. **Depends on:** WP00. **Priority:** P0.

Build manifest validation, direct process launch, monotonic timing, randomized paired blocks, per-arm preparation, timeout handling, immutable raw records, metadata capture, and an analyzer. Separate correctness qualification, shipping-mode timing, and diagnostic profiling. Add a draft mode that refuses to issue confirmatory claims with placeholder pins.

Deliverables: `bench/harness/`, campaign schema, sample schema, deterministic analysis tests, result-card generator, and benchmark self-test fixtures.

Acceptance: known delays inside every measured boundary appear in elapsed time; ratio-direction and paired-unit tests pass; bad manifests and incomplete samples are rejected; failed candidates remain in the denominator. The harness can compare two non-PT commands without importing PT.

Measurement: quantify harness-only overhead with trivial processes, but do not subtract arbitrary estimated overhead from primary end-to-end results.

WP02 — Corpus, environment locks, and reset procedures

Owner: test infrastructure engineer. **Depends on:** WP00, WP01. **Priority:** P0.

Create mechanism probes, adversarial projects, and a representative repository shortlist. Pin qualified revisions, interpreter artifacts, dependencies, and services. Establish training/confirmatory/holdout partitions before optimization tuning. Implement deterministic fixture/database resets and explicit bytecode/filesystem-cache procedures.

Deliverables: `bench/corpus/manifest.toml`, environment locks, reset/probe scripts, corpus eligibility report, and initial edit-replay inventory.

Acceptance: two clean materializations produce the expected inventories/outcomes; failures to reproduce are recorded rather than hand-fixed silently. Every scenario has a checkable starting-state vector. At least one application-dominated negative control is included.

Risk: repository installation difficulty can bias selection. Publish excluded candidates and the reason; do not substitute only synthetic no-op tests.

WP03 — Semantic oracle and adverse-case qualification

Owner: compatibility engineer. **Depends on:** WP00, WP01, WP02. **Priority:** P0.

Implement occurrence-aware inventory comparison, phase/outcome normalization, fixture/import sentinels, exit checking, artifact checks, and contract-specific ordering rules. Build independent broken-runner fixtures to ensure equal pass counts cannot conceal omissions. Retain raw events alongside normalized comparisons.

Deliverables: `tests/conformance/`, canonical trace schema, normalizer rules, oracle comparator, and mismatch triage reports.

Acceptance: detect skipped setup/teardown, duplicate/lost item identities, strict-xpass changes, hidden collection failures, missing subprocess coverage, and stale source execution. Only explicitly nondeterministic fields are normalized; paths, order, or IDs are never broadly erased to force a match.

Boundary: finite conformance is reported as tested support, not universal semantic proof. Flaky baseline cases receive a visible uncertainty classification.

WP04 — CLI, packaging, and fresh-process supervisor

Owner: systems engineer. **Depends on:** WP00. **Priority:** P0.

Build the Rust CLI, interpreter resolution, explicit bridge installation checks, entrypoint profiles, no-daemon path, cancellation, output drainage, and owned process cleanup. Preserve cwd, argument boundaries, relevant environment, and signal information. Create a Rust release artifact and small Python bridge package without pytest shadowing or automatic plugin registration.

Deliverables: `crates/pt-cli`, `crates/pt-process`, Python package skeleton, installation smoke tests, and cleanup tests.

Acceptance: repeated invocations use fresh execution PIDs/state; mutations from an earlier run do not survive; cancellation and child crashes cannot yield success; invalid interpreters fail before project execution. Baseline pytest remains behaviorally and operationally unaffected by installation.

Measurement: empty and one-test process-launch overhead, including cleanup, compared with the pinned reference invocation.

WP05 — Conservative real-pytest backend

Owner: pytest integration engineer. **Depends on:** WP03, WP04. **Priority:** P0.

Load the bridge explicitly and observe authoritative collection and reports without changing the standard run loop. Ensure collection and execution happen in the same process/session. Forward plugin arguments, import/capture/warning policies, and configuration behavior. Expose a plan revision without requiring a second collection.

Deliverables: `python/pt_bridge`, backend adapter, supported-version tests, and a serial conformance report.

Acceptance: the mandatory serial corpus matches the reference contract; import-time sentinels prove no hidden collection pass; teardown failures remain failures; `--collect-only`, selection flags, no-tests cases, and plugin options behave as documented. Unsupported introspective plugins are classified explicitly.

Exit gate: this is the first usable milestone. No accelerator is enabled until it passes.

WP06 — Content index and identity engine

Owner: compiler/tooling engineer. **Depends on:** WP00, WP02. **Priority:** P1.

Implement Python candidate indexing, content-addressed file records, provisional imports, environment identity collection, and watcher-assisted invalidation. Keep authoritative collection separate. Handle malformed syntax, unicode paths, symlink policy, duplicate identities, editable installs, and watcher overflow.

Deliverables: `crates/pt-index`, `crates/pt-identity`, identity versioning specification, and index benchmarks.

Acceptance: same-size/same-mtime edits and dependency/config changes invalidate correctly under the stated input contract; a missed index candidate never suppresses a default full-run test. Unknown external identity components disable authoritative reuse.

Measurement: cold scan, warm validation, single-file update, memory footprint, and index write amplification across small and large trees. Include validation cost in product scenarios.

WP07 — Plan, capability policy, and transactional store

Owner: runtime architect. **Depends on:** WP05, WP06. **Priority:** P0/P1.

Implement progressive plans, occurrence identities, capability composition, provenance, schema migrations, cache invalidation, atomic artifact publication, quotas, and cache recovery. Keep live Python objects out of the persisted format. Distinguish hints from validated metadata and reconstructable runtime preparation.

Deliverables: `crates/pt-plan`, `crates/pt-policy`, `crates/pt-store`, decision explanations, and fault-injection tests.

Acceptance: unknown or conflicting capabilities deny optimization; plugin changes revoke evidence; corrupted stores cannot create a passing result; duplicate node IDs retain multiplicity; a cached plan cannot dispatch into an unrelated session.

Measurement: plan creation/validation/read/write cost and storage growth. A cache slower than recomputation is bypassed under a documented policy.

WP08 — Generation integrity and watch control plane

Owner: runtime engineer. **Depends on:** WP04, WP06, WP07. **Priority:** P1.

Implement generation-tagged requests/results, debounce policy, watch cancellation, stale-result suppression, rescan after overflow, and strict snapshot mode for supported path/input contracts. Keep live-tree and atomic-snapshot assurances distinct. Define concurrent-edit behavior and bounded request queues.

Deliverables: `crates/pt-watch`, generation state machine, snapshot adapter, and edit-race tests.

Acceptance: edit-during-run, rapid edits, edit-and-revert, deletion/rename, and late-arriving old results never mark the wrong generation green. Unverifiable live-tree integrity is surfaced. Strict snapshots reject unsupported external paths rather than silently excluding them.

Measurement: edit-to-request, snapshot construction, debounce, invalidation, and edit-to-valid-result latency. Do not measure only execution after watch dispatch.

WP09 — Production event protocol, reporting, and fault handling

Owner: runtime/reliability engineer. **Depends on:** WP04, WP05, WP07. **Priority:** P0.

Harden the minimal bridge protocol with dedicated channels, framing, bounds, sequence validation, backpressure, artifact references, typed terminal outcomes, and user-facing diagnostics. Ensure terminal compatibility mode and structured reporting can coexist.

Deliverables: `crates/pt-protocol`, `crates/pt-report`, protocol fixtures, compatibility negotiation, and fault-injection suite.

Acceptance: lost/duplicate/truncated frames, disk-full writes, bridge death, output floods, and coordinator crashes cannot become false success or unbounded buffering. Required reports are committed before success. Original failed attempts survive retry-plugin reporting.

Measurement: event overhead for tiny-test and output-heavy suites; memory under backpressure; failure-report latency. Shipping-mode overhead is compared with unmodified pytest.

WP10 — Parallel backend and constrained scheduler

Owner: scheduling engineer. **Depends on:** WP03, WP05, WP07, WP09. **Priority:** P1/P2.

First integrate pinned xdist as a backend with explicit resource and scheduling profiles. Then prototype a capability-gated scheduling adapter using duration estimates, legal groups, and resource constraints. Preserve required next-item/finalization behavior through the supported backend interface rather than duplicating it casually.

Deliverables: xdist adapter, scheduling-policy registry, fixed-resource benchmark matrix, and scheduling decision traces.

Acceptance: inventories, worker lifecycle, supported fixture multiplicities, fail-fast behavior, and custom scheduling hooks satisfy the named parallel contract. Unknown ordering constraints fall back. Work stealing never splits an indivisible group.

Measurement: compare default and tuned xdist on the held-out set at equal total resources. Attribute gains to load balance or reduced permitted setup; do not compare only against serial pytest.

WP11 — Audited framework preloading experiment

Owner: systems engineer. **Depends on:** WP03, WP04, WP05, WP07, WP09. **Priority:** experimental P1.

Implement a minimal dedicated Python seed and disposable children on explicitly supported platforms. Define the audited preload list, initialization boundary, environment invalidation, fork-safety checks, thread/resource constraints, and spawn fallback. Project snapshots are not part of this package's default deliverable.

Deliverables: seed prototype, safety contract, invalidation tests, compatibility report, and go/no-go analysis.

Acceptance: no executed child becomes a seed; global-state edits are visible in later runs; disallowed threads/resources deny preload; interpreter/plugin changes rebuild; coverage/import-order cases match the permitted contract. The baseline always remains available.

Measurement: first-run tax, steady-run savings, break-even N, idle memory, copy-on-write growth, and cleanup. Disable the feature by default if benefits do not survive lifecycle accounting.

WP12 — Dependency observation and attribution

Owner: Python instrumentation engineer. **Depends on:** WP03, WP05, WP07, WP09. **Priority:** P1.

Implement provenance-tagged runtime dependencies with explicit scope attribution and declared-input integration. Attribute shared fixture dependencies to all affected consumers. Recognize missing coverage of subprocesses/native code/threads and broaden confidence scope rather than ignoring it.

Deliverables: dependency collector, graph schema, attribution tests, confidence model, and overhead measurements.

Acceptance: fixture/config/data dependencies survive changes; prior failed/missing observations cannot imply irrelevance; unsupported observation channels produce uncertainty. Collector/coverage coexistence is qualified per version or disabled explicitly.

Measurement: full-run instrumentation overhead and training cost, separate from later selection savings. Reject collecting more metadata than the product can justify.

WP13 — Affected selection, replay corpus, and failure audits

Owner: incremental-runtime engineer. **Depends on:** WP08, WP12, WP03. **Priority:** P1.

Implement conservative selection after authoritative collection, previous-failure inclusion, broad invalidation, empty-selection semantics, and explicit affected-run reporting. Build independent full-oracle replay audits and controlled mutations covering hidden/broad dependencies.

Deliverables: selection engine, at least the initial 300-edit evidence target or a visibly narrower report, replay manifest, miss reports, and workflow cost curves.

Acceptance: zero observed missed stable failures on the mandatory supported corpus; broad unknown changes trigger full scope; no selected-run success is rendered as full-suite success. Every audited miss blocks release until fixed or the affected capability is disabled.

Measurement: compare with full pytest and qualified testmon workflows; include training, observation overhead, invalidations, selection size, and break-even. State statistical limits honestly.

WP14 — Managed service/resource leases

Owner: systems/integration engineer. **Depends on:** WP07, WP09, WP10. **Priority:** P2.

Implement endpoint leases, capacities, exclusivity, deadlock prevention, namespace/reset policies, cancellation release, and worker-death recovery. Provide one reference adapter for a controlled local service before expanding integrations.

Deliverables: resource protocol, lease manager, reference adapter, contamination tests, and service-cost scenarios.

Acceptance: no Python connection object is reused across incompatible process boundaries; lease loss is visible; resource limits hold during crashes; data reset prevents cross-run contamination. Unsupported resource semantics disable reuse.

Measurement: cold service start, fresh isolated service, reusable-reset service, idle resources, and concurrent capacity. Claims identify service reuse separately from runner speed.

WP15 — Coverage and ecosystem qualification

Owner: compatibility engineer. **Depends on:** WP03, WP05, WP09; WP12 for coexistence tests. **Priority:** P0/P1.

Qualify established coverage integration and the initial plugin matrix. Include async execution, property-based testing, database fixtures, custom collectors, retry plugins, and reporting plugins where selected for support. Record effective coverage core and fallback behavior.

Deliverables: versioned integration matrix, coverage set comparator, subprocess/merge tests, and documented exclusions.

Acceptance: required file/line/arc/context artifacts match the stated contract; omitted coverage never counts as acceleration; generation and finalization failures cannot publish a complete report. Plugin upgrades invalidate prior qualification until rerun.

Measurement: identical coverage/reporting configurations across arms; separate no-coverage, line, branch, and context cases.

WP16 — Platform hardening, security, and endurance

Owner: reliability engineer. **Depends on:** WP04, WP07, WP08, WP09, WP15. **Priority:** P0.

Complete process cleanup, path/encoding behavior, cache permissions, IPC authentication, disk quotas, concurrent-client ownership, and platform-specific accounting. Stress long watch sessions, version upgrades, daemon crashes, resource exhaustion, and abandoned stores. Review cache and protocol inputs as untrusted data without claiming test sandboxing.

Deliverables: platform matrix, threat-model review, endurance results, leak analysis, and recovery documentation.

Acceptance: mandatory scenarios pass on each claimed platform; no escaped owned children in supported non-hostile cases; bounded event/store memory; old or corrupt state cannot generate success. Unsupported native capabilities fall back cleanly.

Measurement: whole-session peak and idle memory, CPU while idle, handle/file-descriptor growth, and latency drift over long replays.

WP17 — Confirmatory performance campaign and independent review

Owner: performance lead, with a non-author reviewer. **Depends on:** WP01–WP03, WP05, WP09, WP15, WP16, plus every feature enabled in the candidate. **Priority:** release gate.

Freeze the candidate and campaign. Run the preregistered matrix, compatibility qualification, feature ablations, paired analysis, resource accounting, and held-out replays. Keep failures/exclusions visible. Reproduce critical findings on an independent machine/session and audit the public claim against raw evidence.

Deliverables: immutable campaign bundle, raw samples, correctness reports, analysis outputs, regression inventory, and signed/reviewed decision record.

Acceptance: sections 12–20 are satisfied; no current correctness blocker; all release gates in section 22 have a decision of pass, fail, or inconclusive. Inconclusive is not rounded into pass.

Stop condition: if gains vanish against the strong baseline or only exist on synthetic no-op tests, do not ship a broad speed claim. Ship a narrower capability or remove the optimization.

WP18 — Native executor feasibility, not a V1 prerequisite

Owner: research/runtime engineer. **Depends on:** WP03, WP07, WP17 profiling evidence. **Priority:** research.

Identify a measured Python orchestration bottleneck that persists after simpler changes. Define a restricted execution contract and prototype only that surface behind a feature flag. Keep the real-pytest backend as a differential reference. Do not intercept `pytest` imports or broaden support claims by skipping incompatible tests.

Deliverables: bottleneck evidence, subset specification, prototype, conformance deltas, maintenance-cost estimate, and go/no-go ADR.

Acceptance: a meaningful held-out end-to-end gain under equal work, no unsupported semantic shortcuts, and clear diagnostics/fallback before effects begin. The proposal fails if speed requires weakening ordinary fixture/plugin behavior outside its declared subset.

Decision: stop or defer if expected user benefit does not justify a second semantic implementation. V1 delivery does not depend on this research succeeding.

WP19 — Release, documentation, and ongoing regression controls

Owner: release engineer. **Depends on:** WP17 and all selected release features; not WP18. **Priority:** P0.

Publish installable artifacts, supported matrix, default-safe configuration, result-scope semantics, benchmark reproduction instructions, capability explanations, known limits, and rollback controls. Add smoke conformance per change, scheduled full conformance, and dedicated performance runs on stable hardware.

Deliverables: release checklist, user/developer docs, artifact provenance, regression dashboards, and reproducibility bundle.

Acceptance: a clean machine can install and run the supported profiles without changing the ordinary pytest environment; documented commands match actual CLI behavior; release claims can be regenerated from pinned evidence. Public regressions are not hidden by replacing the benchmark corpus.

Operational rule: semantic regressions disable the offending optimization first. Performance regressions are triaged with a recorded baseline and reproducible scenario rather than noisy single-run alarms.

22. Dependency roadmap and release gates

22.1 Milestones

Milestone	Required packages	Exit decision
M0: Definitions and measurement	WP00–WP03	Contracts, corpus, and harness can reject broken candidates
M1: Trustworthy runner	WP04–WP05, WP09 core	Same-session pytest execution and adverse-case parity
M2: Explainable incremental infrastructure	WP06–WP08, WP09 complete	Identity, plans, generation integrity, and explanations
M3: Measured preparation/ parallel improvements	WP10–WP11	Each optimization earns its own enablement decision
M4: Useful affected feedback	WP12–WP13, WP15	Audited selection with honest scope and amortization
M5: Release qualification	WP16–WP17, WP19	Reproducible correctness and performance evidence
Optional extension	WP14	Qualified external-resource reuse
Independent research	WP18	Native subset justified by measured user benefit

The benchmark and conformance tracks begin before optimization. WP04 can run in parallel with corpus/ oracle work. WP10 and WP11 can proceed independently after their prerequisites. WP14 and WP18 are not allowed to become hidden requirements for delivering the conservative product.

22.2 Proposed numerical gates

Gate	Proposed rule	Interpretation
G-Correctness	Zero unresolved mismatches on mandatory claimed profiles	Any false success is a release blocker
G-Generation	Zero false current-generation success in adverse edit tests	Older successful work is not current proof
G-Wrapper	Upper confidence bound on mean paired excess over `max(5% of baseline time, 20 ms)` is at most zero on designated serial core scenarios	Proposed overhead budget; not an achieved result
G-Feature	Lower confidence bound exceeds 1.05× and absolute saving meets a scenario-specific preregistered threshold	Required before enabling an optional accelerator by default
G-Regressions	No supported key scenario with a confirmed slowdown over both 10% and 50 ms, unless explicitly accepted and disclosed	Aggregate gains do not hide large regressions
G-Selection	No observed missed stable failure on the mandatory replay corpus; adequate stated diversity	Does not establish zero population miss probability
G-Resources	Fits the declared CPU/memory budget; no unbounded growth or leaked owned workers	Speed does not excuse resource escape

Gate	Proposed rule	Interpretation
G-Reproducibility	Independent rerun reproduces direction and practical conclusion on the claimed profile	Exact numerical equality is not required

Freeze absolute thresholds per scenario after the pilot and before confirmatory measurement. The 5%, 20 ms, 10%, and 50 ms values are initial product-policy proposals. Revise them through an ADR, not after seeing a preferred answer. A scenario with a wide interval remains inconclusive; allocate more measurement or keep the feature opt-in.

Zero unresolved correctness mismatches is strict for **claimed** profiles, not a requirement to support every plugin ever written. Unsupported cases must be discovered and reported before unsafe optimization, and remain in coverage denominators.

22.3 Definition of done for any optimization

An optimization is done only when its capability and invalidation rules are documented; correctness and failure tests pass; direct and strong baseline comparisons are complete; attribution and resource costs are known; first-run and steady-state behavior are reported; default enablement is justified; and users can explain or disable it. Code completion alone is insufficient.

23. Risk register and kill criteria

Risk	Detection	Response
Bridge changes hook/import behavior	Differential corpus and plugin inspection cases	Narrow support, adjust integration, or use passthrough
More planning duplicates pytest work	Import sentinels and startup/collection attribution	Keep same-session resolution; remove speculative preflight
Invalid environment/source cache	Content adversaries and edit replays	Downgrade to hints; expand validation or reject reuse
Unsafe fork/preload	Threads/resource probes and platform conformance	Disable capability; retain fresh spawn
Fixture/order semantic drift	Multiplicity/order/side-effect oracle	Preserve backend policy or require explicit contract
Selection misses failures	Full-oracle mutation/replay audits	Block release and broaden invalidation
Instrumentation costs exceed savings	Full-run and sequence break-even	Make observation opt-in, sample safely, or remove it
Hidden memory/ background CPU	Whole-boundary accounting and endurance	Bound caches/services; disable persistence
Corpus overfitting	Frozen holdout, training split, independent reproduction	Narrow claims or restart confirmation
Statistical confidence is overstated	Dependence, effective N, raw time-order review	More independent sessions or an inconclusive result
Unknown plugins dominate fallback	Real-corpus coverage denominator	Prioritize useful adapters, not a broad rewrite

Risk	Detection	Response
Native executor becomes a compatibility sink	WP18 maintenance/benefit review	Stop research; retain pytest semantic backend

Project-level kill criterion: after the conservative architecture and two well-chosen optimization experiments, if representative workloads show no practically useful latency or workflow gain against strong baselines, do not continue a Rust rewrite merely because it is aesthetically appealing. Preserve useful indexing/explanation components and reconsider the product boundary.

24. Decisions still requiring implementation evidence

The following are intentionally open: exact supported version matrix; whether the initial bridge can remain observational across the chosen plugin set; the audited framework preload list; whether any collection-runtime reconstruction is worth supporting; source snapshot behavior for absolute-path applications; instrumentation choice for dependency attribution; the first external-resource adapter; and user-appropriate idle-memory budgets.

These are bounded work items, not permission to make ambiguous claims. Each has an owner in section 21 and a conservative default: fresh real-pytest execution, no authoritative collection reuse, no unsafe preloading, no invisible subset selection, and no hidden service reuse.

Appendix A. Repository layout and core records

A.1 Proposed repository layout

```
crates/
  pt-cli/          pt-process/      pt-protocol/
  pt-index/        pt-identity/     pt-plan/
  pt-policy/       pt-store/        pt-watch/
  pt-report/       pt-scheduler/
python/
  pt_bridge/
tests/
  conformance/     fault_injection/ integration/
bench/
  harness/         corpus/          manifests/
  resets/          replays/         analysis/
docs/
  adr/             contracts.md    benchmark-charter.md
```

Keep the standalone benchmark harness independent of PT's runtime modules. Corpus artifacts and large raw measurements can live in a separate versioned artifact store, referenced by digest.

A.2 Essential typed records

```
RunRequest:
  request_id, argv, cwd, invocation_profile, interpreter_identity
  requested_scope, generation_identity, generation_assurance
  backend, parallel_policy, resource_budget, required_artifacts

ExecutionPlan:
  plan_id, schema_version, request_id, state
  environment_identity, source_identity, collection_identity
  item_occurrences[], groups[], constraints[], capability_decisions[]
  provenance[], owning_session_id?, policy_version

PhaseReport:
  run_id, session_id, occurrence_id, attempt_id
  phase, outcome, reason, was_xfail?, strict_xpass?
  monotonic_start?, monotonic_end?, clock_domain
  output_artifacts[], exception_artifact?, sequence

FinalResult:
  run_id, generation_identity, generation_status, scope
  selected_count, executed_count, deselected_count
  phase_outcome_counts, incomplete_items[], collection_errors[]
  backend_exit_code?, infrastructure_status, cleanup_status
  required_artifacts[], capability_summary, terminal_sequence
```

A final-result validator cross-checks terminal accounting with the occurrence inventory. Report field absence distinctly from zero. Do not assume every selected item produces a call report: setup skip/failure can legitimately prevent a call, while teardown may still matter.

Appendix B. Benchmark scenario template

The companion `benchmark-campaign.template.yaml` is a **non-executable draft** until exact artifacts, corpus entries, expected outcomes, and scripts are supplied and validated. It is not a set of fabricated benchmark results. The following fields illustrate the necessary level of specificity:

```
schema_version: 1
status: draft
campaign_id: null
hypothesis: "Equal-work serial overhead stays within the agreed budget"
claim_family: equal_work
primary_endpoint: request_to_complete_valid_result
candidate:
  revision: null
  binary_sha256: null
baseline:
  kind: pytest_console
  environment_lock_sha256: null
scenario:
  source_revision: null
  scope: full
  workers: 1
  cache_state:
    coordinator: absent
    framework_seed: absent
    pt_metadata: absent
    python_bytecode: valid
    filesystem_cache: warm-procedure
  generation_assurance: immutable_snapshot
  coverage: none
  expected_inventory_sha256: null
experiment:
  paired_blocks: 30
  order: balanced_randomized
  random_seed: 42017
  sample_unit: complete_invocation
  analysis_unit: independent_paired_block
  warmup_policy: declared_state_preparation_only
  bootstrap_resamples: 10000
publication:
  require_correctness_qualification: true
  preserve_failed_and_excluded_samples: true
  allow_confirmatory_with_unresolved_fields: false
```

A production manifest additionally resolves commands, machine limits, reset/probe scripts, installed versions, reporting settings, exclusion rules, and the planned analysis. The included schema and checklist are starting contracts for the harness, not a claim that semantic validation can be accomplished by JSON Schema alone.

Appendix C. Sample and result-card layouts

C.1 Raw sample

Each raw sample records campaign/scenario/block/arm IDs; observed order; session/host IDs; artifact identities; start/end monotonic readings; wall time; CPU/memory metric scope; exit/signal; expected/observed work digests; generation and cleanup status; stdout/stderr/artifact digests; reset/probe evidence; and validity/exclusion reason.

`invalid`, `timeout`, `wrong_result`, `crash`, `unsupported`, and `valid_expected_outcome` are distinct statuses. A nonzero exit can be valid for an intentional failing-test scenario; an unexpected zero exit is a wrong result.

C.2 Human-readable result card

```
Campaign / scenario:         [resolved IDs]
Claim family / contract:     [equal-work, amortized, or affected]
Candidate / baseline:       [revisions and artifact digests]
Corpus eligibility:         [qualified / total, with exclusions]
Requested / executed work:  [identities, counts, and scope]
Initial state / resources:  [full cache vector and budget]
Independent units:          [paired blocks, sessions, hosts]
Wall time:                  [baseline and candidate, interval]
Speedup and absolute delta: [estimator and interval]
CPU / peak / idle memory:   [values and metric definitions]
Correctness / coverage:     [qualification and effective settings]
Missed failures:            [affected mode only, with denominator]
Regressions and failures:   [all relevant cells]
Decision:                   [pass / fail / inconclusive]
Evidence bundle:            [digest and reproduction instructions]
```

All bracketed values are placeholders. No timing or success count should be populated until observed.

Appendix D. Primary references and source boundary

References were consulted on 7 November 2026. They support descriptions of existing tools and relevant runtime behavior. The architecture, work packages, thresholds, and experimental governance in this document are proposed engineering requirements, not claims made by those sources. Web pages and default branches are mutable; a real benchmark must pin artifacts and archive the relevant documentation/revisions.

[S01] pytest-rs project. Project scope and runner approach.
https://github.com/Yasu-umi/pytest-rs

[S02] rpytest project. Persistent/warm-runtime approach and project documentation entrypoint.
https://github.com/neul-labs/rpytest

[S03] rupt project. Rust-oriented test-runner approach.
https://github.com/bobtholamew/rupt

[S04] pytest-xdist, "How it works." Worker collection, collection agreement, scheduling, and report forwarding.
https://pytest-xdist.readthedocs.io/en/stable/how-it-works.html

[S05] pytest-xdist, "Running tests across multiple CPUs." Distribution policies including grouping and work stealing.
https://pytest-xdist.readthedocs.io/en/stable/distribution.html

[S06] pytest, "Fixtures reference." Fixture lifetimes, visibility, and dependency semantics.
https://docs.pytest.org/en/stable/reference/fixtures.html

[S07] pytest, "Writing hook functions." Hook behavior and wrappers.
https://docs.pytest.org/en/stable/how-to/writing_hook_functions.html

[S08] pytest, "Writing plugins." Plugin discovery, explicit loading, entry points, and local plugins.
https://docs.pytest.org/en/stable/how-to/writing_plugins.html

[S09] pytest, "How to invoke pytest." Invocation differences and repeated in-process invocation caveats.
https://docs.pytest.org/en/stable/how-to/usage.html

[S10] Python 3.14, multiprocessing documentation. Process start methods and fork/platform restrictions.
https://docs.python.org/3.14/library/multiprocessing.html

[S11] pytest, "Exit codes." Distinct completion and error statuses.
https://docs.pytest.org/en/stable/reference/exit-codes.html

[S12] testmon documentation. Dependency-informed test selection and initial dependency collection.
https://www.testmon.org/

[S13] pyperf, "Run a benchmark." Reproducibility, system tuning, and benchmark configuration.
https://pyperf.readthedocs.io/en/latest/run_benchmark.html

[S14] pyperf, "Analyze benchmark results." Raw samples, metadata, instability, and result inspection.
https://pyperf.readthedocs.io/en/latest/analyze.html

[S15] ASV, "Tuning timing measurements." Time-scale variation, interleaving, host tuning, and native threading.
https://asv.readthedocs.io/en/latest/tuning.html

[S16] Linux kernel, cgroup v2 documentation. Hierarchical resource accounting, memory peak, and CPU interfaces.
https://docs.kernel.org/admin-guide/cgroup-v2.html

[S17] coverage.py, configuration reference. Measurement cores, contexts, concurrency, and feature restrictions.
https://coverage.readthedocs.io/en/latest/config.html

[S18] hyperfine project documentation. Command timing, warming, preparation, and version-specific invocation behavior.
https://github.com/sharkdp/hyperfine

Final engineering principle: optimize only work that has been shown to be unnecessary under an explicit contract. Prove that the intended work still happened, and start the clock before the work you claim to have removed.